General Notes

- You will submit a minimum of three files, the core files must conform to the following naming conventions (including capitalization and underscores). 123456789 is a placeholder, please replace these nine digits with your nine-digit Bruin ID. The files you must submit are:

  1. *123456789_stats102a_hw4.R*: An R script file containing all of the functions you wrote for the homework. The first line of your .Rmd file, **after loading libraries, should be sourcing this script file**.
  2. *123456789_stats102a_hw4.Rmd*: Your markdown file which generates the output file of your submission.
  3. *123456789_stats102a_hw4.html/pdf*: Your output file, either a PDF or an HTML file depending on the output you choose to generate.
  4. *included image files*: You may name these what you choose, but you must include all of the image files you generated for your structured flowcharts, otherwise your file will not knit. One way to attach an image file is to use ![*title*](*Images/example1.png*).
  5. Please place all of your files into a single folder named 123456789_stats102a_hw4 and compress the folder into 123456789_stats102a_hw4.zip.

  If you fail to submit any of the required core files you will receive ZERO points for the assignment. If you submit any files which do not conform to the specified naming convention, you will receive (at most) half credit for the assignment.

- Your .Rmd file must knit. If your .Rmd file does not knit you will receive (at most) half credit for the assignment.
  The two most common reason files fail to knit are because of workspace/directory structure issues and because of missing include files. To remedy the first, ensure all of the file paths in your document are relative paths pointing at the current working directory. To remedy the second, simply make sure you upload any and all files you source or include in your .Rmd file.

- Your coding should adhere to the tidyverse style guide: https://style.tidyverse.org/.

- All pseudocode or flowcharts should be done on separate sheets of paper, but be included, inline as images, in your final markdown document.

- Any functions/classes you write should have the corresponding comments as the following format.

```
my_function = function(x, y, ...){
#A short description of the function
#Args:
#x: Variable type and dimension
#y: Variable type and dimension
#Return:
#Variable type and dimension
Your codes begin here
}
```

NOTE: *Everything* you need to do this assignment is here, in your class notes, or was covered in discussion or lecture.

- **DO NOT** look for solutions online.
- **DO NOT** collaborate with anyone inside (or outside) of this class.
- Work **INDEPENDENTLY** on this assignment.
- **EVERYTHING** you submit **MUST** be 100% your, original, work product. Any student suspected of plagiarizing, in whole or in part, any portion of this assignment, will be **immediately** referred to the Dean of Student's office without warning.

1: Dealing with Large Numbers

To simulate how the computer handle the computation with large floating point numbers we define pqnumber objects to keep large numbers. This object should have four integer components. The first one is either $+1$ or -1. It gives the sign of the number. The second and third are p and q. And the fourth component is an integer vector of $p+q+1$ integers between zero and nine. For example, we can use the following object x to keep the number 87654.321.

```
x <- structure(list(sign = 1, p = 3, q = 4, nums = 1:8), class = "pqnumber")
```

(a) Write a constructor function, an appropriate predicate function, appropriate coercion functions, and a useful print() method. This question accounts for 25% of this assignment.

- The **constructor** takes the four arguments: **sign**, **p**, **q**, and **nums**. Then check if the arguments satisfy all requirements for a pqnumber object. If not, stop with an appropriate error message. If yes, return the object.
- A predicate is a function that returns a single **TRUE** or **FALSE**, like is.data.frame(), or is.factor(). Your predicate function should be is_pqnumber() and should behave as expected.
- A useful **print()** method should allow users to print a pqnumber object with the decimal representation defined as follows:

$$x = \sum_{s=-p}^{q} x_s \times 10^s$$

Thus p = 3 and q = 4 with nums = 1:8 has the decimal value

$$0.001 + 0.02 + 0.3 + 4 + 50 + 600 + 7000 + 80000 = 87654.321$$

Please add an argument **DEC=TRUE** to allow the function to print the decimal value. By default, DEC should be FALSE and print all the components of a pqnumber object. E.g.,
sign = 1
p = 3
q = 4
nums = 1 2 3 4 5 6 7 8

- Please create three pqnumber objects for the following numbers to demonstrate is_pqnumber(), print():
  (a) sign = 1, p = 3, q = 4, nums = [1 2 3 4 5 6 7 8], and the decimal value = 87654.321
  (b) sign = 1, p = 6, q = 0, nums = [3 9 5 1 4 1 3], and the decimal value = 3.141593
  (c) sign = -1, p = 5, q = 1, nums = [2 8 2 8 1 7 2], and the decimal value = -27.18282
- A coercion function forces an object to belong to a class, such as as.factor() or as_tibble(). You will create a generic coercion function **as_pqnumber()** which will accept a numeric argument x and return the appropriate pqnumber object. For example, given the decimal number 3.14 with p = 3 and q = 4, the function will return a pqnumber object with num = c(0, 4, 1, 3, 0, 0, 0, 0). You should also create a generic as_numeric() function and a method to coerce a pqnumber object to be a numeric vector. You may use the provided example to showcase your functions.

To sum up, you are expected to provide the following functions.

- pqnumber(sign, p, q, nums)
- is_pqnumber(x)
- print(x, DEC)
- as_pqnumber(x, p, q)
- as_numeric(x)

(b) Addition and Subtraction

- Write an addition function **add(x, y)**[1] and a subtraction function **subtract(x, y)**[2]. Suppose we have two positive pqnumber objects x and y. The function to calculate the sum of x and y is computed as the following decimal representation.

$$x + y = \sum_{s=-p}^{q} (x_s + y_s) \times 10^s$$

. However, because we can have $x_s + y_s > 9$, this decimal representation cannot be used directly.

- So we need a **carry-over** function that moves the extra digits in the appropriate way. Same as one would do when adding two large numbers on paper. Also we need a special provision for overflow, because the sum of two pqnumber objects can be too big to be a pqnumber[3].
- Likewise, a subtraction function should have a **borrowing** function that borrows 10 in the same way as you would do a subtraction with pencil-and-paper.
- Your functions should work for both positive and negative pqnumber objects. Both functions should return a pqnumber object with enough p and q to carry the result.
- Your function should handle the overflow and underflow by returning an appropriate message.
- This question accounts for 60% of this assignment.

[1] add(x, y) = x + y
[2] subtract(x, y)=x - y
[3] Note that the length of a vector in R is stored as signed integers.

(c) Multiplication

Use your add() function to write a multiplication function **multiply(x, y)** which can multiply two pqnumber objects x and y. Think about how you would multiply two large numbers by hand and implement that algorithm in R for two pqnumber objects. The function should also return a pqnumber object. Both functions should return a pqnumber object with enough p and q to carry the result. This question accounts for 15% of this assignment.

You may demonstrate your three functions using the examples provided in (a).

Note: Please ensure you attach the flowcharts or algorithms for your carry-over, addition, subtraction, borrowing, and multiplication functions. Additionally, note that the provided test cases are for your reference only. During grading, we will use different arguments and objects to test your functions. Hence, strive to make your functions efficient, accurate, and robust.